

C++ Comments

The Complete Professional Guide to Clean Code

Platform: CodeHive Academy

Series: C++ Development Fundamentals

Difficulty: Beginner to Intermediate

1. The Philosophy of Commenting

"Code is for the machine, comments are for the human."

In high-stakes software development, comments are your primary insurance policy. They transform a cryptic sequence of logic into a narrative that any developer can follow.

Why professional CodeHive developers use them:

Complexity Management: Tricky algorithms often look like "magic" until they are explained line-by-line.

Temporary Debugging: Instead of deleting code you might need later, you "comment it out" to isolate bugs.

Team Collaboration: In a team environment, your coworkers shouldn't have to play detective to identify what your function does.

Self-Memory: You will forget why you wrote a specific logic 3 months from now. Your comments speak to your future self.

2. Single-line Comments (//)

Single-line comments are the "Post-it Notes" of the programming world. They are brief, focused, and immediate.

The Syntax

The compiler ignores everything from the `//` marker to the very end of that specific line.

```
#include <iostream> using namespace std; int main() { // This comment
explains the purpose of the line below cout << "Welcome to CodeHive!"
<< endl; int score = 100; // Standard inline comment style return
0; }
```

When to use them:

- Describing a specific variable choice.
- Explaining a one-line logic jump.
- Making a short note to fix something quickly.

3. Multi-line Comments (/* ... */)

Sometimes, a short note isn't enough. For documentation blocks, legal headers, or broad logic descriptions, we use Multi-line comments.

The Block Structure

```
/* CodeHive Programming Academy Module: comments.cpp Version: 1.0.4  
Description: This program demonstrates how to use large blocks of  
text to document software intent. */
```

The Logic Pause (Disabling Code)

This is a favorite technique among debuggers. You can wrap large chunks of code in these markers to prevent them from running without having to delete them.

```
cout << "Feature A is active" << endl; /* cout << "Feature B is  
buggy, we are keeping it here"; cout << "until the patch update next  
Monday."; */
```

4. Advanced Commenting: Level 2

As you progress, you should start commenting **Units of Work** rather than just lines of code.

Documenting Functions

At CodeHive, we believe every function should have a "contract" comment that defines its Inputs and Outputs.

```
// Calculates the total price including tax // Parameters: subtotal  
(double), taxRate (double) // Returns: The final price with tax added  
double calculateTotal(double sub, double tax) { return sub + (sub *  
tax); }
```

Descriptive Variables

Helpful comments clarify the "why" of a value:

```
int retryLimit = 3; // API standard only allows 3 attempts int  
timeout = 5000; // 5 seconds in milliseconds
```

5. Pro Annotations: TODO & NOTE

Professional C++ development often involves "tags" that modern IDEs can search for instantly.

// TODO: This tells the editor (and your team) that a feature is incomplete or needs to be revisited later.

// NOTE: Use this to highlight a non-obvious piece of information that is critical for the program to function.

```
int main() { // TODO: Add database connection logic here cout <<
"Initializing CodeHive App ..." << endl; // NOTE: The port must be
8080 for sandbox testing int port = 8080; return 0; }
```

6. Common Mistakes to Avoid

More comments aren't necessarily better. Bad comments can be worse than no comments at all.

Outdated Comments: If you change the code but not the comment, you are lying to the next developer.

Commenting the Obvious: `int i = 0; // set i to 0`. This clutters the screen and provides zero value.

Cluttered Code: Too many single-line comments can make the logic hard to track. Use block comments for overhead info.

The CodeHive Rule: If the code is simple enough to understand at a glance, skip the comment. Focus on the tricky parts.

7. Final Master Example

Let's look at how all these pieces fit together into a professional CodeHive source file.

```
#include <iostream> using namespace std; // Project: CodeHive Point Tracker //
Author: Lead Developer @ CodeHive // Purpose: Multi-comment demonstration
int main() { // 1. Initialize user state
int points = 500; bool isPremium = true;
/* Calculation Logic: Premium users get a 2x multiplier Standard users get 1x */
if (isPremium) { points *= 2; // Double the score }
cout << "Total Points: " << points << endl; // TODO: Implement point persistence to file
// NOTE: Always clean up stream buffers before return
return 0; }
```

CodeHive Best Practices Checklist

- Keep it simple. Write for humans.
- Use English for international team collaboration.
- Be consistent with your // TODO formatting.
- Always update comments during code refactoring.

Thank you for learning with CodeHive Academy.

© 2026 CodeHive Academy. All Rights Reserved.